Secrets
ABAP
Ansible
Apex
AzureResourceManager
C
C#
C++
CloudFormation
COBOL
CSS
Dart
Docker
Flex
Go
HTML
Java
JavaScript
JCL
Kotlin
Kubernetes
Objective C
PHP
PL/I
PL/SQL
Python
RPG
Ruby
Scala
Swift
Terraform
Text
TypeScript
T-SQL
VB.NET
VB6
XML

Dart static code analysis

Unique rules to write Clean DART Code

All rules 115 | Bug 15 | Code Smell 100

Tags | Impact | Clean code attribute | Search by name...

Class names should comply with a naming convention
Code Smell

Track uses of "TODO" tags
Code Smell

Deprecated code should be removed
Code Smell

Cyclomatic Complexity of functions should not be too high
Code Smell

Constant names should comply with a naming convention
Code Smell

Unnecessary nullable in final declaration should be removed
Code Smell

Dependencies should be sorted
Code Smell

Unused assignments should be removed
Code Smell

Standard outputs should not be used directly to log anything
Code Smell

Variables should not be initialized with "null"
Code Smell

Files should end with a newline
Code Smell

Unnecessary nullable in final declaration should be removed

Analyze your code

Intentionality - Clear | Maintainability

Code Smell | Major

A `final` or `const` variable that is declared as a nullable type, but then initialized with a non-null value, should rather be declared as a non-nullable type.

Why is this an issue? | How can I fix it? | More Info

Change the type of the variable to be non-nullable and remove all the null-safety measures potentially used at every call site, such as

- the null-aware operator `?.`
- the null assertion operator `!`
- or any construct introduced to narrow the type

Code examples

Noncompliant code example

```
final int? i = 42; // Noncompliant
final String? s = 'Hello'; // Noncompliant

void main() {
  print(i!);
  print(s?.length);
}
```

Compliant solution

```
final int i = 42;
final String s = 'Hello';

void main() {
  print(i);
  print(s.length);
}
```

Noncompliant code example

```
class AClass {
  static final int? i1 = 42; // Noncompliant
  static final String? s1 = 'Hello'; // Noncompliant

  static void aFunction() {
    final int? i2 = 42; // Noncompliant
    const int? i3 = 42; // Noncompliant

    print(i1!);
    print(s1?.length);
    print(i2 ?? 0);
    print(i3 != null ? i3 : '<no value>');
  }
}
```

Compliant solution

```
class AClass {
  static final int i1 = 42;
  static final String s1 = 'Hello';

  static void aFunction() {
    final int i2 = 42;
    const int i3 = 42;

    print(i1);
    print(s1.length);
    print(i2);
    print(i3);
  }
}
```

Available In:
sonarcloud